

One project. One Render service.

NuruWomen is presented to visitors as Nuru Commons, an English and Kiswahili women's health knowledge prototype for Kenya and wider African use.

Project explanation and deployment guide
Prepared for Samuel and the engineering team | 3 October 2026

Release status: synthetic demonstration. The combined build runs the interface, gateway and analysis service together. It does not provide approved clinical answers, verified clinicians, real patient intake or public health publication.

What the project is trying to solve

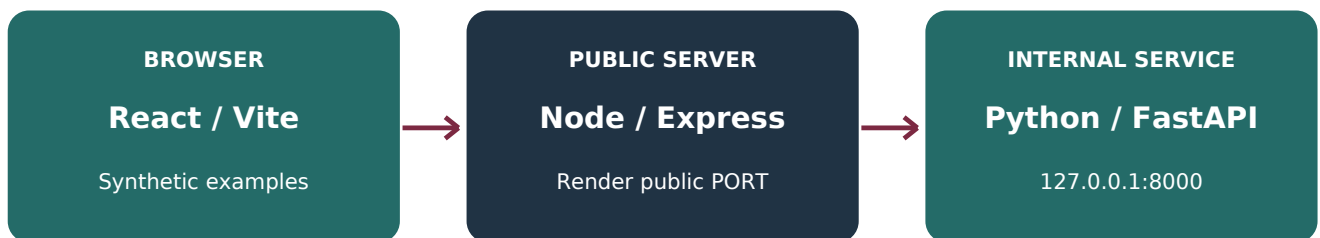
Sensitive health questions often sit between personal experience, professional education and research evidence. NuruWomen proposes a shared space where those forms of knowledge remain clearly labelled and where recurring education gaps can guide future content work.

The final package brings the engineers' contributions into a consistent running application. It preserves the existing visual design, adds native views for research references and imported coverage data, and corrects mismatches between the frontend, backend and AI demonstration contract.

Initial topic	What the prototype covers
Menstrual health	Periods, cramps, cycle changes and related sample subtopics
Sexual health	Sexual and reproductive health vocabulary and example questions
Healthy ageing	Menopause, perimenopause and midlife health examples
Postpartum	After-birth recovery and related health questions
Mental health	Mental wellbeing and example safety cues

Review boundary: this work used the supplied ZIP snapshot. It did not fetch GitHub main or establish that the snapshot contains later commits.

How the application fits together



One Docker container. One browser origin. No public Python endpoint.

The browser receives the built React app from the Node gateway. Analysis requests go to the same website origin under `/api/ai`. The gateway validates the payload, attaches a server-only key and calls Python on the container's loopback interface. Responses keep the original AI JSON contract.

Component	Responsibilities
React frontend	Navigation, sample questions and answers, draft library, research catalogue, coverage view, local privacy cues, and runtime validation of AI responses.
Node gateway	Public listener; SPA assets and route fallback; JSON validation; 64 KiB request limit; per-IP throttling; origin checks; safe errors; upstream deadlines.
Python service	Privacy pattern suggestions, safety cue matching, five-topic routing, synthetic classifier, draft evidence retrieval and translation-review checks.
Process supervisor	Starts Python, waits for initialization, starts Node, handles shutdown and exits if a required child process stops.

Request routes in this build

Public route	Behavior
<code>/api/health</code>	Checks Python availability as well as the public gateway.
<code>/api/ai/nuru/analyze</code>	Accepts a synthetic title/body and requested response language; returns privacy, safety, routing, review and evidence fields.
<code>/api/ai/translation-check</code>	Checks a synthetic translation draft and its source version; never approves publication.

The Python component also has internal endpoints for privacy, classification, retrieval and aggregate experiments. These are not exposed as public gateway routes. No external generative AI provider or database is needed for this demonstration.

What a visitor can do

Page	Working behavior
Home /	Explains the concept and presents a synthetic three-layer example.
Questions /questions	Lists bundled examples and locally saved synthetic questions.
Ask /ask	Runs local pattern checks and server analysis in English or Kiswahili, then saves an invented example in tab memory.
Question /question/:id	Shows sample or local answers in distinct knowledge layers. Synthetic responses and helpful reactions stay local.
Draft library /library	Eleven inherited educational sample articles, search and area filters, source links and Markdown export.
Research /research	Searchable and topic-filtered catalogue of 20 imported references from 2020 onward.
Coverage /blind-spots	Five-topic summary and selectable country details from the data scientist's CSV snapshots.
About /about	Explains implemented behavior, intended design and remaining work.

The invented-question journey

1. Enter a fictional question, then select English or Kiswahili analysis.
2. Inspect the privacy cues, topic suggestions and any safety warning.
3. Run the server check. A failed, cancelled or stale result is not treated as a pass.
4. If identifying patterns are detected, apply a redacted draft, review it and run the check again.
5. Confirm that the example is invented, choose a topic and save it locally.

The AI always returns `publication_allowed: false`. The interface respects that boundary. It does not send questions, replies or votes to public Nostr relays. Reloading the tab or inactivity cleanup removes local demo events.

Language scope

The analysis accepts English and Kiswahili examples and returns language-specific messages and available draft cards. Most interface text and inherited articles remain English. This is not a fully translated application.

What the AI contribution actually does

The AI component is a constrained assistance prototype, not a medical chatbot. Its pipeline combines pattern matching, explicit bilingual topic cues and a small classical text classifier.

Stage	Implementation and limit
Privacy suggestions	Patterns detect examples of Kenyan phone numbers, email addresses, labelled identifiers, name phrases, handles and location phrases. They can miss identifiers and indirect personal stories.
Safety cues	Rule matching can flag wording for a potential urgent concern. A lack of a match does not mean a question is medically safe. The rules have no clinical validation.
Five-topic routing	Explicit bilingual cues suggest broad categories and an abortion subtopic where applicable. Mixed or ambiguous pregnancy-loss wording requests human routing.
Text classifier	TF-IDF features and logistic regression trained on synthetic examples provide topic suggestions and decision scores. Scores are not calibrated clinical probabilities.
Evidence retrieval	Selects exact-language cards. Approval checks inspect content digest/version and clinical-review validity; Kiswahili approval also depends on translation review and a valid source card.
Human-review contract	Responses require human topic/review decisions. This release reports synthetic mode and blocks publication.
Translation check	Reviews draft version consistency and selected language cues. It requires human language and clinical review; it does not translate or approve content.

Two evidence collections, two different purposes

The Python service's `ai/data/evidence_cards.json` contains retrieval demonstration cards. The separate `ai/evidence/` collection contains research metadata and bibliographic links. Adding a journal reference does not make a draft card approved, and the reference catalogue is not a live retrieval system over paper full texts.

The internal synthetic gap-board experiment counts consented question submissions, not people or disease prevalence. It suppresses cells below 25 and restricts complementary small cells. It is not connected to the public coverage page; suppression alone is not an anonymity guarantee.

Research and data-science contributions

The supplied evidence collection contains 20 reference records compiled on 2 October 2026. The frontend now reads a build-generated copy of this manifest and provides title/journal search, five-topic filtering and HTTPS source links. The original index and BibTeX file remain in the project.

Research topic	Reference records
Menstrual Health	4
Healthy Aging	4
Postpartum	4
Sexual Health	4
Mental Health	4

No full-text papers are bundled. Bibliographic metadata was imported from the supplied collection; it was not independently reverified during this code review. References carry their original publisher access conditions. App summaries still require separate clinical review.

Imported country coverage

Topic	Countries	Coverage	Records
Healthy ageing	53 / 53	100%	3,498
Mental health	53 / 53	100%	3,498
Postpartum	53 / 53	100%	2,067
Sexual health	30 / 53	56.6%	60
Menstrual health	29 / 53	54.7%	29

There are 265 country/topic detail rows across five topics and 53 countries. The summary identifies WHO GHO for four topics and a hand-extracted static menstrual-health proxy for ages 15-24. No extraction code, indicator definitions or full reproducibility record was supplied. The dashboard marks these as unverified snapshots and does not infer clinical quality from the numeric coverage levels.

Canonical files and synchronization

Root data/ holds the original summary/detail CSVs. Before each build, scripts/sync-data.mjs validates dimensions and paper dates/links and regenerates the frontend JSON. The standalone HTML dashboard is retained for reference but is not deployed.

What was combined, corrected and removed

Review finding	Final implementation
Duplicated Render frontend/gateway hosting with fixed service URLs	One Docker service and one browser origin. Node serves the app and talks to loopback Python.
Browser permits publication although the AI blocks it	Local memory-only demo events; no public relay publishing.
Synthetic clinician personas and articles appear clinically verified	Demo personas and unreviewed drafts are labelled explicitly. The live clinician trust set is empty.
An arbitrary answer can self-label as clinical	Clinical classification requires trusted authorship. A self-label cannot confer professional authority.
Standalone data dashboard disconnected from React	Coverage CSVs rendered natively in the app, with source status and definition limits shown.
Gateway health succeeds independently of AI readiness	Public health route checks Python. The supervisor handles child-process failures and shutdown.
Database variables exposed as VITE-shaped configuration	Replaced with a server-only runtime template. The SQL proposal is not treated as an implemented backend.
Script CSP allows inline execution/eval; NIP-19 placeholder views	Restricted script policy, expression-free animation engine, and supported note/event routing; unsupported identifier views return 404.

Cleanup with contribution preservation

Removed 57 unreferenced TypeScript scaffold modules, unused agent/editor/MCP configuration, alternate lockfiles and generated dependency/cache/build directories from the delivery. Unused UI packages were removed from the root dependency list. Existing tests, AI development scripts, license notices and source evidence files remain.

The proposed MariaDB/MySQL schema and independent HTML dashboard were moved under docs/source-contributions/. They are historical designs, excluded from the runtime image. The SQL contains destructive table-reset statements and has not been connected or executed.

The archive contains the final source folder and deployment documents. Original engineers' commit history should be preserved in GitHub when applying this consolidation. The local review commit is not a push to the team repository.

Deploy the final folder on Render

The deployment assumes the final folder's contents become the GitHub repository root. The root must contain Dockerfile, render.yaml, package.json, src/, server/, ai/, data/ and scripts/.

Recommended deployment sequence

1. Extract the ZIP and review its README and integration report.
2. Put the final folder contents at the root of a GitHub review branch. Confirm the diff, commit and push. Keep secrets, dependencies and generated output out of Git.
3. In Render, choose New > Blueprint, connect the repository and choose that branch.
4. Render reads render.yaml and creates one Docker web service. Its configuration generates a server-only WHC_API_KEY.
5. Wait for the build and health checks, then open the assigned URL.
6. Check /api/health and exercise the routes using invented examples.

Setting	Included configuration
Service type / runtime	Web Service / Docker
Dockerfile / context	./Dockerfile / repository root
Process command	Docker CMD runs python scripts/start.py; leave override blank
Public port	Render PORT, consumed by Node; image default 10000
Internal Python	127.0.0.1:8000; not exposed as a public service
Health path	/api/health; requires gateway and Python availability
Service key	WHC_API_KEY; generated by Blueprint, or set manually to a random 32+ character ASCII token
Plan	Free demonstration plan in the supplied Blueprint

If you create a service manually, choose Web Service and Language Docker, use the same Dockerfile and health path, and set WHC_API_KEY. A Static Site alone cannot run the Python analysis or gateway.

Deployment verification remains pending. Docker is unavailable in this workspace, so the actual container image was not built here and no Render service was created. Local application builds and the combined runtime were tested. Free Render services sleep after inactivity and have usage limits; measure memory/startup behavior after deployment.

Official Render references: render.com/docs/docker, render.com/docs/blueprint-spec, render.com/docs/health-checks and render.com/docs/free. Checked 3 October 2026.

Verification and honest release limits

Check	Result
Frontend TypeScript, ESLint and production build	Passed
Existing frontend tests	5 tests passed across 3 files
Gateway TypeScript and build	Passed
Existing Python tests	106 passed
Combined runtime HTTP smoke checks	15 checks passed
Local request flow	English/Kiswahili analysis, non-synthetic request rejection, foreign-origin rejection and translation review contract checked
Routing and process behavior	SPA refresh responses, readiness, graceful shutdown and required-child failure checked
Browser visual/workflow automation	10 checks passed: catalogue search, coverage, blocked/save/reload flow, Kiswahili analysis, mobile width, no relay connections or runtime errors
Docker image and hosted Render deployment	Not performed: Docker unavailable; no Render account action taken

The HTTP checks validate service connections and response contracts. The browser checks cover the core demonstration flow, not every possible interaction. Neither check set establishes clinical accuracy or readiness for patient use.

Known non-fatal test output

The existing frontend smoke test emits a React `act()` warning from `NostrLoginProvider`. Python tests emit a `Starlette/httpx` deprecation warning. These do not fail the existing suites. Keep dependency upgrades and test-harness maintenance in the team backlog.

What is still missing

A real clinical review panel and credential-verification process; signed approval records for published content; human moderation and escalation workflows; production consent/privacy rules; durable question/answer storage; end-to-end multilingual UI review; and reproducible research-data extraction. Payments, wallets, private consultations and implemented database RBAC are not part of this release.

The research references support future clinical review, but bibliographic inclusion does not certify the app. The public dashboard describes source-record coverage, not real users, diagnoses or prevalence. App-level logs avoid question text, but hosting infrastructure can process network metadata.

Team ownership and the next build

Team role	Recommended ownership
Frontend engineer	Question/library/research routes, clear draft labels, responsive and accessible browser behavior.
Backend engineer	Gateway contracts, origin/body limits, upstream errors, future persistence and moderator APIs.
Full-stack engineer	Docker/Render deployment, process supervision, health monitoring and complete user-flow verification.
AI/ML engineer	Topic routing and classifier evaluation, privacy/safety cue behavior, retrieval approval gates and model-card maintenance.
Data scientist	Coverage indicator definitions, reproducible extraction scripts, missing-data checks, language/topic coverage and aggregate methodology.
Cybersecurity engineer	Nostr threat model, future auth/trust boundaries, privacy leak testing, secrets, logging and deployment review.
UI/UX designer	Comprehensible demo/review status, language choices, consent wording, mobile layouts and user research.

Practical next steps

First: deploy the synthetic build, verify it in a real browser on desktop and mobile, and share the deployment URL with the team.

Next: resolve content/data provenance, review every draft clinically and linguistically, and agree on consent, moderation and persistence before enabling public health questions.

Then: integrate an authenticated, genuinely verified clinician registry and deliberate Nostr publication controls. Re-evaluate the privacy model before collecting or publishing real health data.

Maintaining one source of truth

Keep canonical research metadata in `ai/evidence/` and country coverage in `data/`. Build synchronization produces the React data views. Update contracts deliberately across `ai/app/schemas.py`, `server/src/schemas/aiRequests.ts` and `src/lib/nuru/aiClient.ts`. Run the existing checks after changes and refresh the validation report.

Licensing and reuse

The AI component's original LICENSE and NOTICE are retained. The supplied snapshot did not contain a clear repository-wide license for all contributions. Confirm contributor permission and agree on a shared license before describing the complete repository as open source. Papers and publisher material retain their own rights.

Useful handoff documents

README.md, docs/DEPLOY_RENDER.md, docs/INTEGRATION_REVIEW.md, docs/VALIDATION.md, ai/docs/MODEL_CARD.md and ai/docs/TRANSLATION_WORKFLOW.md. The root guide is authoritative for this consolidated deployment; AI-only instructions remain component-development references.